ANDROID DEVELOPMENT GUIDE

RecyclerView

Efficiently display large, scrollable datasets in Android with view recycling, adapters, and layout managers

Core classes
RecyclerView, ViewHolder, Adapter, and LayoutManager

Layout managers
Linear, Grid, and Staggered Grid arrangements

Also covers
Kotlin code, CardView items, and edge-to-edge display

Level
Beginner to intermediate Android developers

Based on the official Android Developers RecyclerView guide and a GeeksforGeeks Kotlin walkthrough, with additional facts on performance, DiffUtil, and Compose alternatives.

OVERVIEW

What Is RecyclerView?

- **RecyclerView** is a `ViewGroup` for efficiently displaying large sets of data.
- You supply the data and define how each item looks — RecyclerView creates elements only as they're needed.
- As the name implies, it **recycles** item views: a view scrolled off-screen is reused for new items, not destroyed.
- Result: better performance, smoother responsiveness, and lower power consumption.

> **Good to know:**
>
> - Part of **AndroidX**: `androidx.recyclerview.widget.RecyclerView`
> - Added via `implementation("androidx.recyclerview:recyclerview:1.3.2")` in `build.gradle`
> - Modern successor to the older, now-legacy `ListView` and `GridView`

RecyclerView

The ViewGroup added to your layout that hosts and scrolls the list.

ViewHolder

Wraps a single item's View and caches references to its child widgets.

Adapter

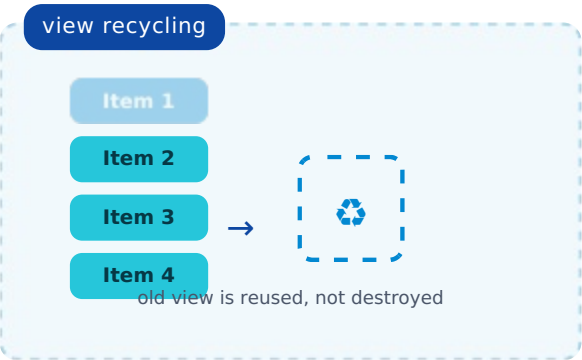
Creates ViewHolders and binds your data to them on request.

LayoutManager

Positions items — linearly, in a grid, or in a staggered grid.

1 How View Recycling Works

- When an item scrolls off one edge, RecyclerView doesn't discard its view
- The off-screen view goes to an internal recycling pool
- It's re-bound with fresh data as a new item scrolls in on the opposite edge
- This avoids the cost of inflating a brand-new layout for every row

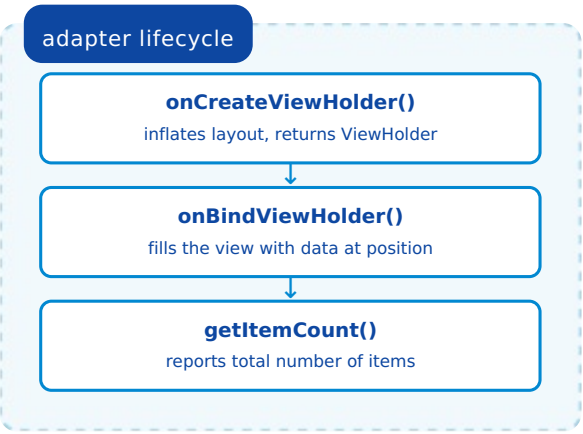


Why it matters:

- Inflating a new View for every row would be slow and memory-hungry
- Only enough views to fill the screen (plus a small buffer) are ever created
- Scrolling stays smooth even with thousands of items

2 Adapter & ViewHolder Methods

Your `Adapter` subclass overrides three key methods that RecyclerView calls automatically as the user scrolls.



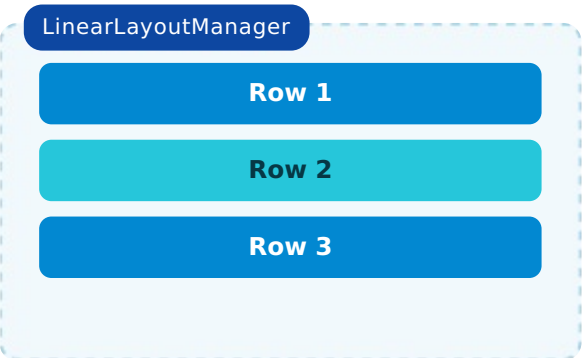
- **`onCreateViewHolder()`** — creates & initializes a new ViewHolder and its View (no data yet)
- **`onBindViewHolder()`** — associates that ViewHolder with data at a given position
- **`getItemCount()`** — tells RecyclerView the dataset size

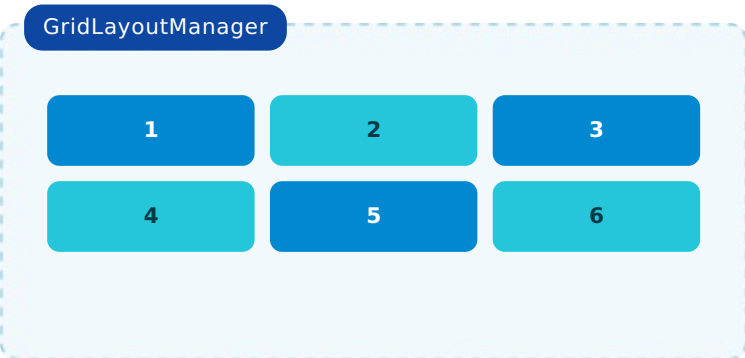
Common beginner mistake:

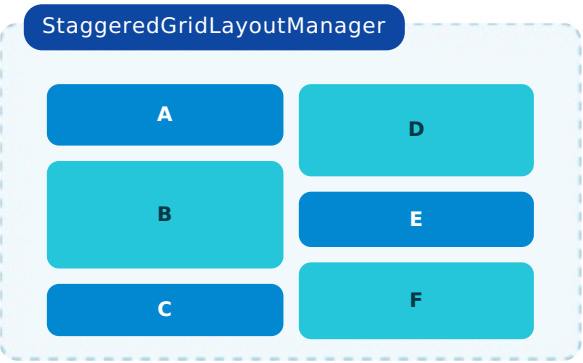
- A RecyclerView with no `layoutManager` set will show nothing
- It's not optional — set it in code, or via `app:layoutManager` in XML

3 Layout Managers

A `LayoutManager` positions the items inside the RecyclerView. The library ships three standard implementations, all extending the abstract `RecyclerView.LayoutManager` class.







Choosing one:

- `LinearLayoutManager` — single vertical or horizontal list
- `GridLayoutManager` — rows & columns of equal size
- `StaggeredGridLayoutManager` — grid without forcing equal heights/widths (Pinterest-style)

4 Steps to Implement RecyclerView

The official Android guide breaks implementation into three core steps:

Step	What you do
1. Plan the layout	Decide how the list or grid should look — usually by picking one of the standard LayoutManagers.
2. Design the item & ViewHolder	Design each list item's XML layout, then extend `RecyclerView.ViewHolder` to hold references to its views.
3. Define the Adapter	Extend `RecyclerView.Adapter` to connect your data source to the ViewHolder views.

5 Adapter Example (Kotlin)

A minimal adapter and nested ViewHolder that binds a list of `Item` objects (an image and a text label) to a card layout, adapted from GeeksforGeeks' Kotlin walkthrough.

```
class Adapter(private val list: List<Item>) :
    RecyclerView.Adapter<Adapter.ViewHolder>() {

  // creates a new ViewHolder when needed
  override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
    val view = LayoutInflater.from(parent.context)
        .inflate(R.layout.card_view_design, parent, false)
    return ViewHolder(view)
  }

  // binds one item's data into the ViewHolder
  override fun onBindViewHolder(holder: ViewHolder, position: Int) {
    val item = list[position]
    holder.imageView.setImageResource(item.image)
    holder.textView.text = item.text
  }

  override fun getItemCount(): Int = list.size

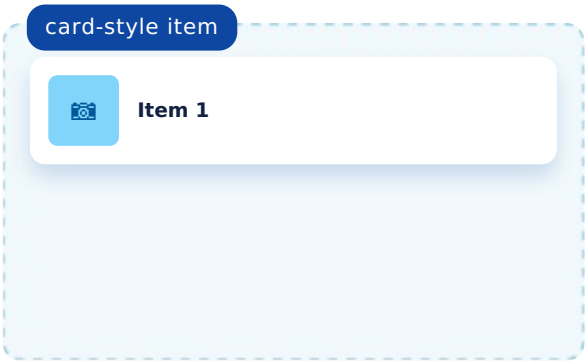
  class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {
    val imageView: ImageView = itemView.findViewById(R.id.imageview)
    val textView: TextView = itemView.findViewById(R.id.textView)
  }
}
```

Why a nested ViewHolder:

- `findViewById()` is relatively expensive to call repeatedly
- ViewHolder looks up each child view **once**, when the item view is created
- References are cached — no repeated lookup every time the item is bound

6 RecyclerView & Item XML

Add <RecyclerView> to your activity layout like any other view, and design a separate layout file for a single row — commonly wrapped in a MaterialCardView for a card-style list.




```
<androidx.recyclerview.widget.RecyclerView
  android:id="@+id/recyclerview"
  android:layout_width="match_parent"
  android:layout_height="match_parent"/>
```

> **Tip:** the `tools:listitem="@layout/card_view_design"` attribute lets Android Studio's layout preview render sample rows at design time, without affecting the app at runtime.

7 Wiring It Up in MainActivity

Once your Adapter and ViewHolder are ready, connect everything to the RecyclerView instance in your Activity or Fragment.

```
val recyclerview: RecyclerView = findViewById(R.id.recyclerview)
// arranges items in a vertical list
recyclerview.layoutManager = LinearLayoutManager(this)
val data = ArrayList<Item>()
for (i in 1..20) {
  data.add(Item(R.drawable.gfg_logo, "Item $i"))
}
val adapter = Adapter(data)
recyclerview.adapter = adapter
```

> **Updating data later:**
>
> - Avoid `notifyDataSetChanged()` for small changes — it rebinds the whole list and disables item animations
> - Prefer targeted calls: `notifyItemInserted()`, `notifyItemRemoved()`
> - Best option: back the adapter with **`ListAdapter`** + **`DiffUtil`**, which computes minimal changes automatically

8 Edge-to-Edge Display

Since Android 15 (API 35) enforces edge-to-edge by default, a RecyclerView's first and last rows can end up drawn underneath the status bar or navigation bar unless insets are handled.



Fix:

- Call `enableEdgeToEdge()`
- Apply system bar insets as padding via `ViewCompat.setOnApplyWindowInsetsListener`
- Set `android:clipToPadding="false"` so items can scroll under the bars without being obscured

9 RecyclerView vs. ListView

Aspect	ListView (legacy)	RecyclerView
View reuse	Manual, via `convertView`	Enforced automatically by the framework
ViewHolder pattern	Optional convention	Required — built into the API
Layout arrangement	Vertical list only	Linear, Grid, or Staggered Grid
Item animations	Manual to implement	Built-in via `ItemAnimator`
Dividers / decoration	Manual drawing	`ItemDecoration` API

Where things stand:

- Google recommends `LazyColumn` / `LazyRow` in Jetpack Compose for new projects
- RecyclerView remains the standard for apps still using the View system

10 Advantages & Disadvantages

Advantages

- Efficiently handles large or infinite datasets
- Built-in support for lists, grids, and staggered grids
- Reduces power consumption via view recycling
- Extensible with ItemDecoration and ItemAnimator

Disadvantages

- More boilerplate than a simple static layout
- No built-in click listener — must wire it up manually
- Requires understanding several cooperating classes at once

SUMMARY

Key Takeaways

- RecyclerView efficiently displays large, scrollable datasets by **recycling** item views instead of recreating them
- **RecyclerView** — the container view added to your layout
- **ViewHolder** — wraps and caches one item's view
- **Adapter** — creates ViewHolders and binds data to them
- **LayoutManager** — arranges items linearly, in a grid, or in a staggered grid

Exam tip: anchor your answer on these four classes and their roles:

- **RecyclerView** — the container view
- **ViewHolder** — wraps and caches one item's view
- **Adapter** — creates ViewHolders and binds data via `onCreateViewHolder()`, `onBindViewHolder()`, and `getItemCount()`
- **LayoutManager** — decides how items are arranged on screen